

Towards Real Time Multi-robot Routing using Quantum Computing Technologies

James Clark^{*}
STFC Hartree Centre
james.clark@stfc.ac.uk

Tristan West
Ocado Technology
tristan.west@ocado.com

Joseph Zammit
Ocado Technology
joseph.zammit@ocado.com

Xiaohu Guo
STFC Hartree Centre
xiaohu.guo@stfc.ac.uk

Luke Mason
STFC Hartree Centre
luke.mason@stfc.ac.uk

Duncan Russell
Ocado Technology
duncan.russell@ocado.com

ABSTRACT

In this paper, we investigate the potential for current quantum computing technologies to provide good solutions to the NP-hard problem of routing multiple robots on a grid in real time. A hybrid quantum-classical approach has been presented in detail. Classical computation is used to generate candidate paths, while quantum annealing is used to select the optimal combination of paths. This second process is generally the most time consuming when performed classically. The performance is benchmarked classically and on a D-Wave 2000Q with up to 200 robots and has shown that producing valid solutions for the problem of multi-robot routing is achievable with the current quantum annealing technology. The current limitations of using quantum annealing are also discussed.

CCS CONCEPTS

• **Computing methodologies** → **Robotic planning; Optimization algorithms**; • **Computer systems organization** → **Quantum computing**; • **Hardware** → **Quantum technologies**; • **Software and its engineering** → **Software performance**;

ACM Reference Format:

James Clark, Tristan West, Joseph Zammit, Xiaohu Guo, Luke Mason, and Duncan Russell. 2019. Towards Real Time Multi-robot Routing using Quantum Computing Technologies. In *Proceedings of HPC Asia 2019*. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Multi-robot routing is a well studied combinatorial optimization problem[4]. There are similar applications in other areas, such as the multiple traveling salesman problem (TSP) and the vehicle routing problem (VRP), which can be applied to surveillance, search and rescue, transportation and component assembly. Solving these problems is computationally expensive and they have been proven to be NP-hard[14]. Many applications of robot routing, TSP, and

VRP require that the problem is solved in real time. In this paper, we are investigating the potential solutions with Quantum Computing technologies.

Quantum Computers are a class of devices that can perform computation by taking advantage of quantum mechanical effects such as entanglement and superposition. These devices operate on qubits [7], which are like classical bits but they can be in a superposition of both 0 and 1 — this means the qubit may show properties of both 0 and 1 at the same time.

Currently, there are two main types of quantum computers: circuit-model and quantum annealing. However, additional models [13] for quantum computation are being researched. Circuit-model quantum computers operate on qubits with quantum gates, which perform transformations on a small number of qubits which are defined by unitary matrices. These quantum gates are analogous to logic gates in traditional electronic circuits. Circuit-model quantum computers have been proven to be universal and have theoretical proofs of ‘quantum speedup’ for certain applications [19]. Current commercial computers have from 10 to 50 qubits. Applications of circuit-model quantum computers include: integer factorization (Shor’s Algorithm) [20], search algorithm (Grover’s Algorithm) [8], simulation of quantum systems [10], and finding approximate results of optimization problems (Quantum Approximate Optimization Algorithm) [5].

Quantum annealing, however, takes a different approach. Quantum annealing evolves a time-dependent Hamiltonian, slowly (via the adiabatic theorem) [6], with the final state encoding a solution to the input problem. ‘Slowly’ in this context is still measured in microseconds.

Although the current available quantum annealers have around 2000 qubits, quantum annealers are not universal quantum computers [12]. This means that quantum annealers cannot be used to run all types of quantum algorithms.

A typical application of quantum annealing is as a metaheuristic to find the global minimum of an objective function. This can be formulated as a quadratic unconstrained binary optimization (QUBO) problem and is shown in Equation (1) [16]. $\mathbf{x}$ is a vector of length L , which represents a list of binary variables and the $L \times L$ matrix Q describes how the variables are related to each other.

$$\min_{x_1, \dots, x_L \in \mathbf{x}} f(\mathbf{x}) = \mathbf{x}^T Q \mathbf{x} \quad (1)$$

^{*}Corresponding author.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

HPC Asia 2019, January 2019, Guangzhou, China

© 2019 Copyright held by the owner/author(s).

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM.

DOI: 10.1145/3293320.3293333

Quantum annealing has already been applied to many different fields, including : traffic flow optimization [18], circuit fault diagnosis [2], modeling of exotic materials [15], training of deep neural networks [1], and in the field of computational biology [17]. In order to create near term applications, a hybrid quantum-classical approach [1, 18, 21, 22] has to be used.

In this paper, we explore how current quantum computing hardware can be used to produce solutions to the problem of routing robots on a grid. This problem is more difficult than the traffic flow optimization problem [18] due to the additional constraint that robots should not collide. The contributions of this paper are the formulation of the robot routing problem as a Quadratic Unconstrained Binary Optimization problem and a demonstration of how an iterable hybrid quantum-classical approach can be used to produce a good set of routes for the robots. We also provide an overview of the state of the art decomposing solver for quantum annealing, and identify the areas which need to be improved for real time multi-robot routing.

2 FORMULATION OF A MULTI-ROBOT ROUTING PROBLEM USING QUANTUM ANNEALING

A multi-robot routing problem is specified by a set of n robots at a location $R = \{r_1, r_2, \dots, r_n\}$ and a set of n target destinations $D = \{d_1, d_2, \dots, d_n\}$. Each point r_i and point d_i is located on a $N \times M$ grid defined by $G = \{g_1, g_2, \dots, g_{N \times M}\}$, i.e. $R \subset G$ and $D \subset G$. Each robot visits its associated target destination via one of many possible routes within G and each route has a cost which is dependent on the distance the robot travels and if the robot makes a change in direction (a turn) in its chosen route. The goal for the multi-robot routing problem is to find a route of minimum cost for each robot to travel from its location r_i to its target destination d_i , while avoiding collisions with all other robots and cells marked out of bounds. In general, there are many possible routes that each robot can take, but a smaller number of optimal or near-optimal routes. The optimal cost for a route that a single robot takes is equal to the Manhattan distance, plus the cost of a single turn (if required).

Routing robots with a quantum annealer currently requires a hybrid quantum-classical approach, where some computation is performed classically on a traditional computer and difficult computation is offloaded to the quantum annealer. Generating simple optimal routes for a single robot is relatively computationally inexpensive, classically, but determining which combination of these routes is optimal when avoiding collisions can be expensive. This approach is based on the traffic optimization work done by Neukart et al. [18]. Figure 1 shows the main steps of the algorithm to route multiple robots, with classical steps in orange and quantum steps in purple.

The robot starting positions and target destinations, along with any additional information required to complete the job, are created via a job generator. This mimics a warehouse queue.

The main areas in which robot routing differs from traffic optimization are: candidate route generation, constraints for building the QUBO problem, and the post-processing.

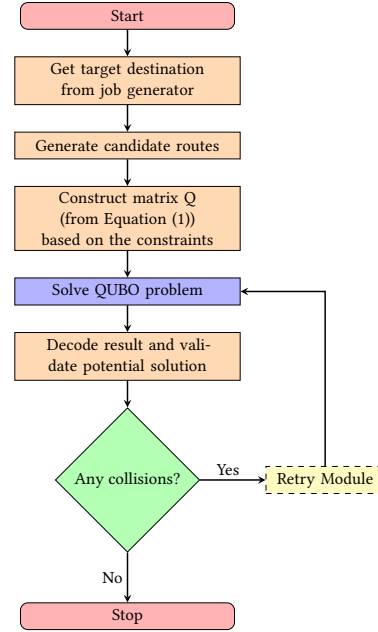


Figure 1: Process of routing multiple robots with a quantum annealer. Orange boxes are classical processes, purple boxes use a quantum process.

2.1 Path Generation

The candidate routes are generated with a modified A^* algorithm, which punishes changing direction, to minimize turns. The A^* algorithm is a best-first search algorithm, commonly used for path finding [9]. In this case, both the robot's position r_i and target destination d_i are passed to the A^* implementation, which generates a number of different routes. $\{j_1, j_2, \dots, j_n\}$ are used to represent the number of candidate routes per robot. A binary variable, q_{ij} where i is the robot index and j is the candidate route index, represents whether or not the route was taken by the robot i . q_{ij} is set to 1 if the route is chosen or 0 if the route is not chosen. In order to reduce the number of binary variables, initially only four candidate routes are generated per robot, however if there is a collision, the number of routes for the colliding robots has to be increased to increase the chance that there is a non-colliding route. The details are shown in Section 2.4. Figure 2 shows an example of the different paths that can be generated for a single robot, while avoiding out of bounds points.

2.2 Multi-robot Routing QUBO Problem with Constraints

After generating paths for all robots, the multiple robot routing optimization problem can be expressed by a QUBO, shown in Equation (3), where the choice of each possible path is represented by a binary variable, stored in $\mathbf{q}$ (Equation 2), each grid point g_k^t for each time step t , and a weighting factor λ .

$$\mathbf{q} = \{q_{11}, q_{12}, \dots, q_{1j_1}, q_{21}, \dots, q_{2j_2}, q_{n1}, \dots, q_{nj_n}\} \quad (2)$$

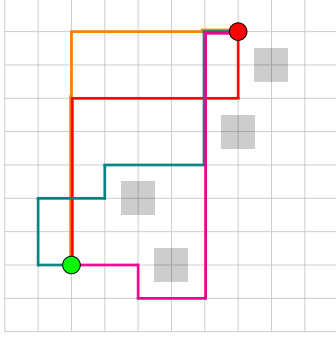


Figure 2: Four paths that are generated for a robot, traveling between the red and green points. Out of bounds points are marked with gray boxes.

$$f(\mathbf{q}) = \sum_k \sum_t \left(\text{cost}(g_k^t) \right) + \lambda \sum_i \left(\sum_j q_{ij} - 1 \right)^2 \quad (3)$$

The coefficients of the binary variables q_{ij} are the coefficients in the matrix Q from Equation (1).

Once the QUBO problem has been created classically, it can be solved with the quantum annealer.

To explain the derivation in Equation (3), we examine it in two parts. Equation (4) represents the constraint for ensuring a robot i has exactly one path assigned to it where the linear coefficients contribute to the diagonal coefficients in Q and the quadratic coefficients contribute to the off-diagonals. This uses the binary rule $q_{ij}^2 = q_{ij}$ to simplify the expression [18]. This is repeated for all robots.

$$\begin{aligned} \left(\sum_{j=0}^{j_i} q_{ij} - 1 \right)^2 &= -q_{i1} - q_{i2} - \dots - q_{ij_i} \\ &+ 2q_{i1}q_{i2} + 2q_{i1}q_{i3} + \dots + 2q_{ij_i}q_{i(j_i-1)} \\ &+ 1 \end{aligned} \quad (4)$$

After all the candidate routes are generated, each grid point will have a record of the candidate routes that travel through it and which time step this occurs.

$$\begin{aligned} \text{cost}(g_k^t) &= \alpha \left(\sum_{q_{ij} \in g_k^t} q_{ij} \right)^2 + \\ &\beta \left(\sum_{q_{ij} \in g_k^{t+1}} q_{ij} \right)^2 + \\ &\gamma \left(\sum_{q_{ij} \in g_k^{t-1}} q_{ij} \right)^2 \end{aligned} \quad (5)$$

In order to avoid collisions, we use a cost function (Equation (5)) that operates on each grid point at each point in time, g_k^t . The value

of this function depends on the binary variables for the candidate routes that travel through the grid point. The first part of the cost function $\alpha \left(\sum_{q_{ij} \in g_k^t} q_{ij} \right)^2$ ensures a higher cost if multiple routes are at the same grid point at the same time (i.e. there is a collision). The second section $\beta \left(\sum_{q_{ij} \in g_k^{t+1}} q_{ij} \right)^2$ performs a 'look-ahead' to see what routes use this grid point during the next time step. This will stop robots from swapping positions to avoid collisions between each time step. Finally, the last part of the cost function $\gamma \left(\sum_{q_{ij} \in g_k^{t-1}} q_{ij} \right)^2$ looks at the previous time step and adds a penalty if there was a robot at that grid point, this is to stop robots following each other too closely. The contribution of each part of the cost function can be controlled by the coefficients α , β , and γ . The cost function will contribute to the diagonal and off diagonal coefficients of matrix Q .

2.3 Validating Chosen Routes and Collisions

The next stage of the algorithm is to decode the result and check for invalid solutions. The result that is returned from the quantum annealer is a bitstring with either 1 or 0, each value representing the candidate routes q_{ij} . For example, for 3 robots with 4 routes each:

001000011000

can be decoded as robot 1 taking route 1, 3, robot 2 taking route 2, 4, and robot 3 taking route 3, 1. The number of routes per robot and total number of robots is tracked for decoding purposes.

After the bitstring is decoded, the solution is validated by ensuring each robot has one route. If any robot has 0 or multiple routes, the solution is marked as invalid. This is checking that the constraint in Equation (4) is not broken.

The next stage of validation is to check the number of collisions that the solution has. Collision detection is completed by checking the chosen routes pairwise and ensuring that no two robots share the same cell or swap positions at any point in time. If there are any violations are found in this process, the solution is marked as invalid.

2.4 Solution for Colliding Robots

If a robot has a collision with the original candidate routes, the chosen routes from the quantum annealer are invalid. By fixing the number of routes for non-colliding robots to 1 (the chosen route), we can use the 'spare' binary variables to generate extra routes for the robots that had collisions. The new problem can then be solved with the quantum annealer. Given enough candidate routes for colliding robots, a valid solution may be found. The drawback to this method is that non-colliding robots are not given the chance to change their path which may result in a suboptimal solution. Figure 3 shows an overview of constructing a new problem based on a previously found invalid solution.

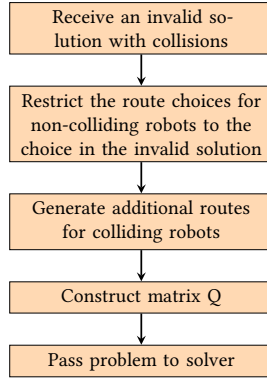


Figure 3: Process of taking an invalid solution and attempting to find a valid solution

3 REAL-TIME MULTI-ROBOT ROUTING USING QUANTUM ANNEALING

3.1 A Note About Embedding

Embedding is the process of mapping the structure of a problem on to the structure of the quantum annealer. The embedding process can be handled manually or by a library call. In this case, the decomposing solver qbsolv will handle embedding the problem. The current hardware available has limited connectivity between the qubits, so chains of physical qubits are created to represent highly connected logical qubits. The largest fully connected graph (clique) that can fit on the D-Wave 2000Q has 64 elements.

3.2 Running Large Problems with qbsolv

In order to run large problems (problems that do not fit on the quantum annealing hardware), a decomposing solver must be used. qbsolv is a decomposing solver for QUBO problems, developed by D-Wave. qbsolv can use a classical solver locally or use a remote quantum annealer [3].

The algorithm for qbsolv involves using a hybrid quantum-classical approach to solving the large problem. First, a random solution is created. Next, an iterative approach is taken: the importance of the variables is ranked and a subproblem (subQUBO) that can fit on the quantum annealer is created based on the importance ranking. The subQUBO is then solved using the annealer. The result from the subQUBO is used to update the global solution and the iterative part of the process is repeated until the global solution no longer improves. The approach uses a classical tabu search for local areas with the quantum annealer performing a global search [3, 18].

3.3 Implementation

The algorithm was implemented using the cloud API provided by D-Wave to access a D-Wave 2000Q quantum annealer.

The code was split in to different classes, roughly following the different computational steps described previously:

- Problem generator (Generate problems for a warehouse queue)
- Path generator (Generates candidate paths for the robots)
- QUBO generator (Applies the constraints to the robots and candidate paths)

- D-Wave solver (Wrapper for communicating with the D-Wave API or local solver)
- Post processing (Performs validation and collision detection)
- Retry module (Retries the problem, if necessary)

3.4 Software and Solver Details

The software used to program the D-Wave 2000Q is shown in Table 1.

Table 1: Software versions used

Software	Version
Python	2.7.14
dwave-qbsolv	0.2.7*
dwave-cloud-client	0.4.1
dwave-system	0.3.1
dimod	0.6.10

* with modifications

qbsolv was modified to return the total time spent running on the QPU, in addition to the result.¹

The solvers used in the Results section are shown in Table 2. The yield refers to the percentage of qubits and couplers that can be used, as compared to the maximum of 2048 qubits and 6016 couplers.

Table 2: Different solvers used to obtain results

Solver	Details
DW_2000Q_2	99% Yield (2040 qubits 5967 couplers)
qbsolv	Classical solver built in to qbsolv, running on an Intel Xeon Gold 6142

4 RESULTS

Due to the non-determinism of quantum annealing, the quantum annealer may not return identical results when an identical problem is run multiple times. To account for this in our evaluation, the same problem was run multiple times. For comparing the performance the average compute time was noted, and when checking the quality of the solution the best solution was taken.

As the path generation was not the focus of this project, the performance for the pathing part of code was not investigated in detail. The pathing for each robot is independent, which may be parallelised to improve performance.

Some of the terms used in the results are described below:

- QUBO Construction — time spent constructing the QUBO problem.
- Classical Solver — time spent in the qbsolv library when running on a classical processor.
- QPU Solver — time spent in the qbsolv library when using a quantum annealer as a solver. This includes embedding the problem, communication to the cloud API, and waiting in the job queue.

¹Available at github.com/jameshclrk/qbsolv

- Embedding — time spent embedding (and unembedding) a problem in qbsolv.
- Communication — time spent sending a job to the remote quantum annealer and waiting in the job queue.
- Quantum Annealing — time spent accessing the quantum annealer, including time to: program the machine, perform the annealing cycle, read the state of the qubits, and post-process the data read from the qubits.

4.1 Test cases for routing multiple robots

The test cases used for testing the implementation involve n robots operating on an $N \times M$ grid. Each robot starts at location r_i is given a destination d_i to travel to, where it may remain at for period of time. Each grid point can hold a maximum of one robot at each possible moment in time. Some points on the grid may be marked as out of bounds. Each robot is randomly assigned its starting position.

The robots are allowed to move by one grid point each time step and only in the X or Y direction. If a robot has to change direction, there is a turning cost. For this problem, the turning cost is implemented by remaining stationary for one time step before continuing.

Figure 4 shows an example problem on a small grid. In reality the problems may be on a larger grid with a large number of robots.

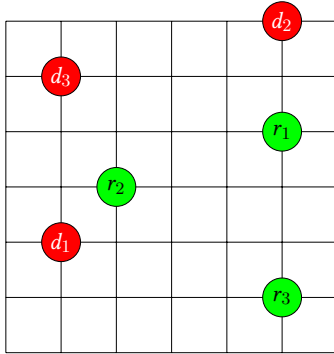


Figure 4: Example problem definition

The test cases were solved using the algorithm described in Section 2.

4.2 Classical Solver

The results shown in this section relate to the classical solver that is included with the qbsolv library, as an alternative to the quantum annealing hardware in order to benchmark the time to solution.

The current bottlenecks for running this problem are the path generation, QUBO construction, followed by the solver. As the number of robots increases, the collision detection execution time starts to dominate as the number of robots increases.

Figure 5 shows the execution time for the QUBO construction, solver and collision detection for a varying number of robots on a fixed grid size. With a smaller number of robots, the solver is the slowest part of the code but the QUBO construction quickly dominates the run time of the code.

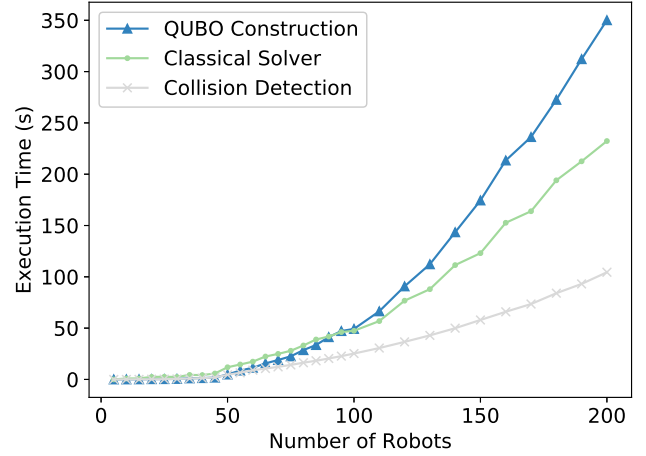


Figure 5: The execution time as the number of robots grows, on a fixed grid size (50×50)

Figure 6 shows how these areas of the code grow as the number of robots increases. Both the solver and QUBO construction are growing at a similar rate, but the collision detection grows faster – this means eventually the collision detection will dominate when the density of robots on a grid becomes sufficiently large.

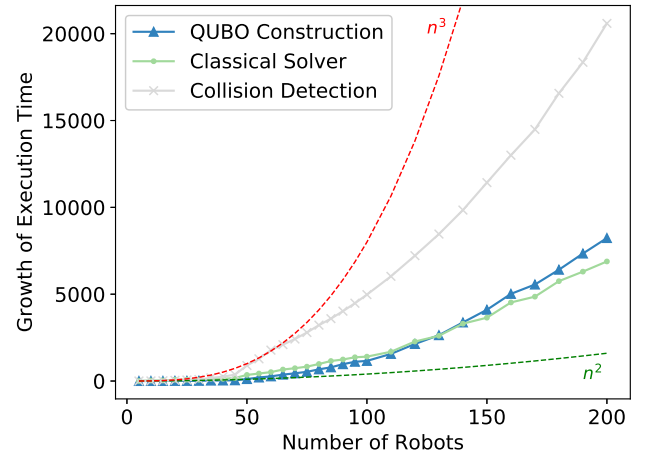


Figure 6: The growth of execution time as the number of robots grows, on a fixed grid size (50×50), as compared to 1 robot

In order to see how out of bounds cells can impact performance, test cases with differing numbers of out of bounds cells were created. The results are shown in Figure 7. The execution time for the solver doesn't seem to be affected by the increase, while the collision detection seems to have a slight increase in execution time. The QUBO construction has the most drastic performance degradation when a large proportion of cells are marked as out of bounds – the execution time for QUBO construction currently depends on the length of the candidate routes. When candidate routes have to

avoid a large number of out of bounds cells, they may become very long.

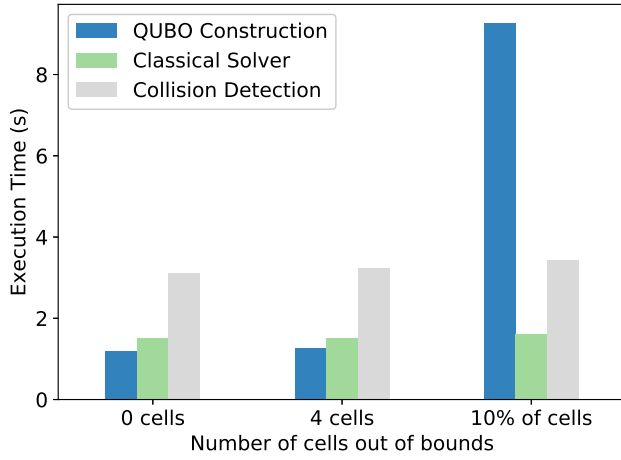


Figure 7: The execution time for different parts of the code for changing numbers of out of bounds cells

4.3 Quantum Solver

This section shows the results of using the quantum annealer to find solutions to the multi-robot routing problem.

Much like the classical solver, the path generation and QUBO construction are bottlenecks when running on the QPU. Each of these stages are identical to the classical solver. However, the solver stage of the code is drastically slower. This is mainly due to the overhead of using a cloud API with a job queue for the remote QPU. When considering only the time spent on the QPU (time spent accessing the quantum hardware, including programming, annealing, and post-processing), the quantum solver can require less time to run. However, this is not a fair comparison as the problem has to be embedded on to the QPU. The performance of the solver is heavily dependent on the connectivity of the graph of the original problem.

qbsolv will solve ‘small’ problems classically, even when asking to use a quantum annealer as a solver. As a result, we do not show the problem with less than 5 robots in the results for the quantum solver.

The execution time for the QPU solver with a breakdown of the execution time is shown in Figure 8. The communication time clearly dominates, as mentioned previously. The next largest section of the execution time is spent embedding the problem. The most intensive part of the embedding happens once, which is finding the largest clique that fits on the current D-Wave solver. There is also an embedding and unembedding cost for each subQUBO, which explains the growth as the problem size increases.

Figure 9 shows the scaling of the solver and QPU access time when running the same problems as the classical solver. The scaling of the solver does not have a smooth increase, but instead has wild fluctuations. This is because qbsolv needed to run more iterations of its algorithm to find a solution, and therefore had to send

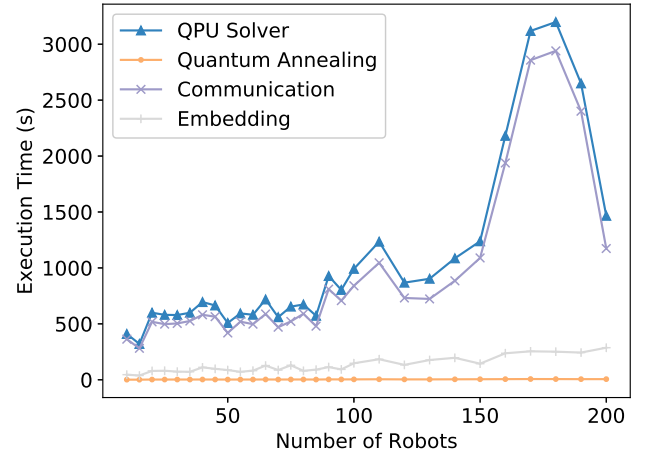


Figure 8: Execution time of the solver QPU version of the code with a breakdown of what components make up the execution time.

more subQUBOs to the quantum annealer. For the test cases that were run, qbsolv with the quantum annealer seems to scale significantly better than qbsolv’s internal classical solver.

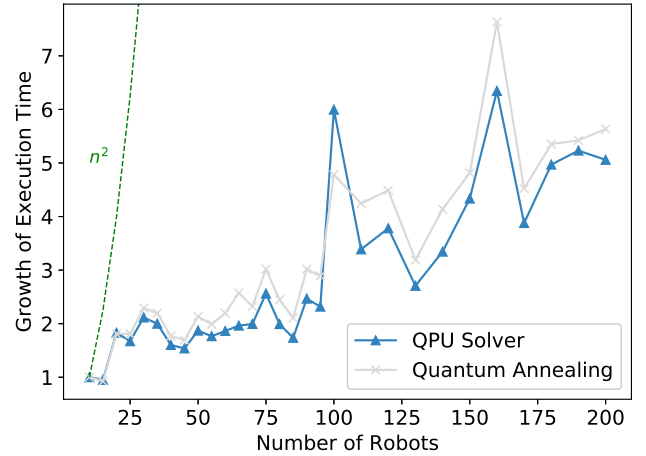


Figure 9: Scaling of the QPU solver time and QPU access time as the number of robots increases, on a fixed grid (50×50), compared to 5 robots

As the number of subQUBOs increases, the number of calls to use the remote solver increases. This means that the execution time for the communication, embedding, and annealing increases linearly as the number subQUBOs increases, as shown in Figure 10. Each data point is a unique run with a problem on the quantum annealer. There is a spike at the end of the communication time, which may be the result of a congested job queue. From the results, it appears that the best way to reduce the execution time is to minimize the number of subQUBOs sent to the quantum annealer. This could be accomplished by a domain specific embedder – which can

take advantage of knowledge about the problem to decompose the QUBO problem – or by using a quantum annealer that has higher connectivity and more qubits.

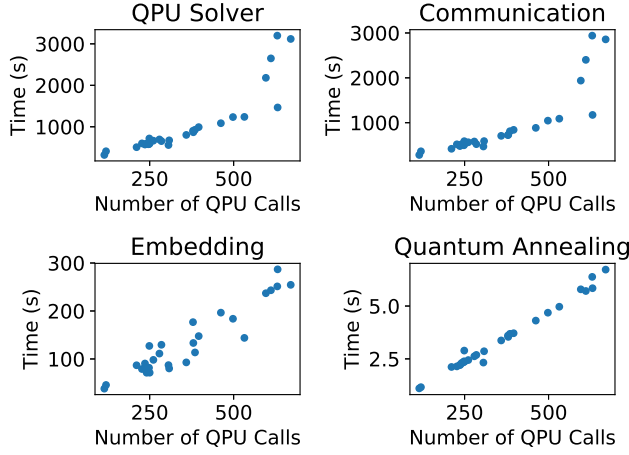


Figure 10: Scaling of the total execution time of different sections of the code compared to the number of subQUBOs.

Figure 11 shows the execution time of the classical solver and the quantum solver when removing communication time. This shows that if you could minimize the time spent waiting for the cloud API and in the job queue, the quantum annealing solver would become competitive with the classical solver, even with the embedding overhead. The embedding process has room for optimization. Considering the future hardware architecture, where more highly connected graphs can be decomposed with fewer QUBOs or even embed natively on to the QPU, Figure 11 also shows the potential for routing a large number of robots in real time, see ‘Quantum Annealing’ time.

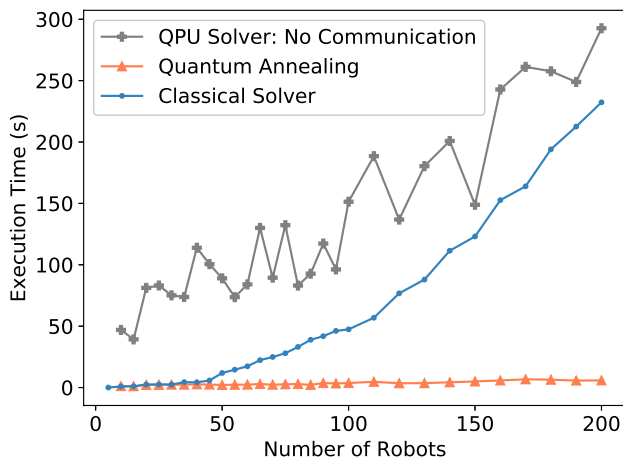


Figure 11: QPU Solver time without communication and queuing time, compared with the classical solver time.

4.4 Performance of Solutions with Colliding Robots

The aim of generating additional routes for colliding robots was to get a better solution than with a fixed number of routes for each robot. A suite of problems was created, with different numbers of robots, grid sizes, and number of out of bounds cells. This suite was run with both additional route generation enabled and disabled to see how the solutions were affected.

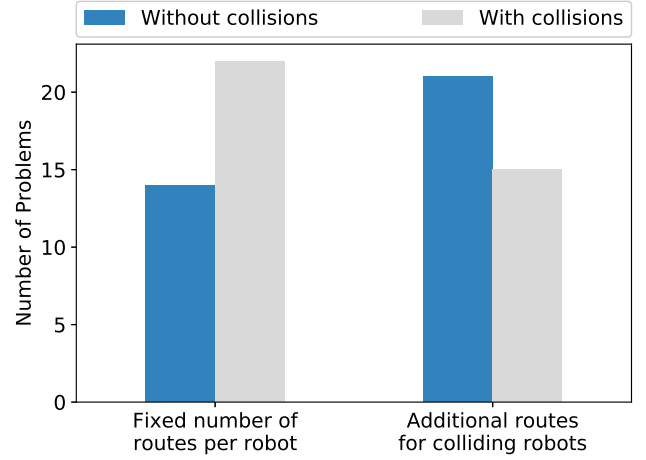


Figure 12: Number of valid and invalid solutions found with and without additional route generation.

Figure 12 shows that generating additional routes is useful in finding valid solutions when the fixed number of routes problem was not able to find a valid solution. There are still a number of invalid solutions even after generating additional routes. Figure 13 shows that even though there were invalid solutions, generating additional routes for collisions tends to outperform the original fixed number of routes solution. There were a number of attempts where retrying did not result in a better solution or give the same performance. This is due to the non determinism of quantum annealing: the attempt with retrying did not happen to find the solution with the lower cost. This shows that at least one of the additional generated routes was selected by the annealer. Increasing the number of additional routes may improve the solution further.

5 CONCLUSIONS AND FUTURE WORK

We have shown that it is possible to route multiple robots on a grid using quantum technology with today’s hardware. The performance of the quantum annealing is promising and has potential for routing a large number of robots in real time. However, today there are performance bottlenecks on both the classical and quantum part of the hybrid algorithm.

The main bottlenecks on the classical side are creating the QUBO problem and collision detection. These areas were not optimized and may be improved in our future work.

The bottlenecks for the quantum solver are the communication time and the embedding time. The communication time could be

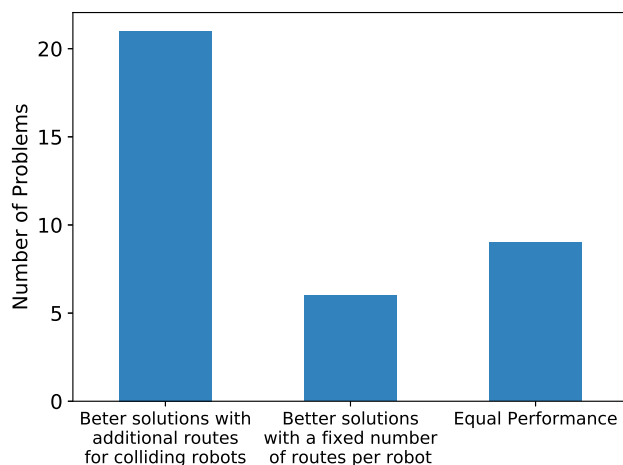


Figure 13: Generating additional routes for test problems with collisions usually resulted in a better solution.

reduced by hosting quantum annealing device locally which is dedicated to routing robots, however this is expensive and hosting a quantum computer requires experience with extremely cold temperatures – the operating temperature for these devices is around 13 mK.

The time spent on the quantum processor is only a small percentage of the run time, so our next focus will be on trying to reduce the embedding time. One approach is to cache the largest clique that qbsolv found so that each new problem does not have to recompute this. This minimizes the impact of the large embedding cost at the beginning of the run, as it is cached after the first problem has been executed.

Another solution to reduce the embedding cost is to create a new algorithm which can use domain specific knowledge to decompose the problem. By introducing more knowledge in to the decomposing algorithm, we expect that the number of subQUBOs will be reduced, resulting in lower overall embedding time. If we tailor this algorithm to take advantage of the architecture of the specific quantum annealer by fitting our problem on to the underlying graph structure of the qubits, we may be able to reduce the embedding overhead further.

One of the main issues with the current generation of quantum annealers is the lack of logical qubits available on the device. This is due to a limited number of physical qubits and connections between them. In general, the number of logical qubits is much less than the available physical qubits. As the hardware architecture matures, the number of physical qubits is expected to grow. However, a change of architecture is required to increase the number of connections per qubit. D-Wave have recently announced a new architecture which increases the number of qubits and the number of connections [11]. A highly connected architecture will mean that larger subQUBOs can be run, which should result in fewer calls to the QPU, reducing execution time.

In future work, we aim to extend the problem definition so that robots are given new destinations after they reach their destination. This would provide a more realistic simulation of a real world applications and may highlight different areas that can be improved.

6 ACKNOWLEDGEMENTS

The work carried out in this paper was funded via the Bridging for Innovators programme through the Science Technology Facilities Council.

REFERENCES

- [1] Steven H Adachi and Maxwell P Henderson. 2015. Application of quantum annealing to training of deep neural networks. *arXiv preprint arXiv:1510.06356* (2015).
- [2] Zhengbing Bian, Fabian Chudak, Robert Brian Israel, Brad Lackey, William G Macready, and Aidan Roy. 2016. Mapping constrained optimization problems to quantum annealing with application to fault diagnosis. *Frontiers in ICT* 3 (2016), 14.
- [3] Michael Booth and Steven P. Reinhardt. 2017. *Partitioning Optimization Problems for Hybrid Classical / Quantum Execution TECHNICAL REPORT*. Technical Report. D-Wave Systems Inc.
- [4] W. Burgard, M. Moors, C. Stachniss, and F. E. Schneider. 2005. Coordinated multi-robot exploration. *IEEE Transactions on Robotics* 21, 3 (June 2005), 376–386. <https://doi.org/10.1109/TRO.2004.839232>
- [5] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028* (2014).
- [6] Edward Farhi, Jeffrey Goldstone, Sam Gutmann, and Michael Sipser. 2000. Quantum computation by adiabatic evolution. *arXiv preprint quant-ph/0001106* (2000).
- [7] Neil Gershenfeld and Isaac L Chuang. 1998. Quantum computing with molecules. *Scientific American* 278, 6 (1998), 66–71.
- [8] Lov K Grover. 1997. Quantum mechanics helps in searching for a needle in a haystack. *Physical review letters* 79, 2 (1997), 325.
- [9] Peter E Hart, Nils J Nilsson, and Bertram Raphael. 1968. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics* 4, 2 (1968), 100–107.
- [10] Matthew B Hastings, Dave Wecker, Bela Bauer, and Matthias Troyer. 2014. Improving quantum algorithms for quantum chemistry. *arXiv preprint arXiv:1403.1539* (2014).
- [11] Mark W Johnson. 2018. Future Hardware Directions of Quantum Annealing. In *Qubits Europe 2018*. Retrieved September 12, 2018 from https://www.dwavesys.com/sites/default/files/mwj_dwave_qubits2018.pdf
- [12] Mark W Johnson, Mohammad HS Amin, Suzanne Gildert, Trevor Lanting, Firas Hamze, Neil Dickson, R Harris, Andrew J Berkley, Jan Johansson, Paul Bunyk, et al. 2011. Quantum annealing with manufactured spins. *Nature* 473, 7346 (2011), 194.
- [13] Stephen P Jordan. 2008. Quantum computation beyond the circuit model. *arXiv preprint arXiv:0809.2307* (2008).
- [14] Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. In *Proceedings of a symposium on the Complexity of Computer Computations, held March 20-22, 1972, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, USA*. 85–103. <http://www.cs.berkeley.edu/%7Eluca/cs172/karp.pdf>
- [15] Andrew D King, Juan Carrasquilla, Jack Raymond, Isil Ozfidan, Evgeny Andriyash, Andrew Berkley, Mauricio Reis, Trevor Lanting, Richard Harris, Fabio Altomare, et al. 2018. Observation of topological phenomena in a programmable lattice of 1,800 qubits. *Nature* 560, 7719 (2018), 456.
- [16] Gary Kochenberger, Jin-Kao Hao, Fred Glover, Mark Lewis, Zhipeng Lü, Haibo Wang, and Yang Wang. 2014. The unconstrained binary quadratic programming problem: a survey. *Journal of Combinatorial Optimization* 28, 1 (2014), 58–81.
- [17] Richard Y Li, Rosa Di Felice, Remo Rohs, and Daniel A Lidar. 2018. Quantum annealing versus classical machine learning applied to a simplified computational biology problem. *NPJ quantum information* 4, 1 (2018), 14.
- [18] F. Neukart, G. Compostella, C. Seidel, D. von Dollen, S. Yarkoni, and B Parney. 2017. Traffic flow optimization using a quantum annealer. *Frontiers in ICT* 4 (2017), 29.
- [19] Michael A Nielsen and Isaac L Chuang. 2000. *Quantum computation and quantum information*. Cambridge University Press.
- [20] Peter W Shor. 1999. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review* 41, 2 (1999), 303–332.
- [21] Olawale Titiloye and Alan Crispin. 2011. Graph coloring with a distributed hybrid quantum annealing algorithm. In *KES International Symposium on Agent and Multi-Agent Systems: Technologies and Applications*. Springer, 553–562.

- [22] Tony T Tran, Minh Do, Eleanor G Rieffel, Jeremy Frank, Zhihui Wang, Bryan O’Gorman, Davide Venturelli, and J Christopher Beck. 2016. A hybrid quantum-classical approach to solving scheduling problems. In *Ninth Annual Symposium on Combinatorial Search*.